

Guia de Arquitetura & Estrutura de Pastas NestJS

Padrão Modular (Feature-First), DDD & Multi-Tenant Isolado para o Backend GLPI / ITSM (Schema v9)

1. Princípios de Arquitetura & Organização em Camadas

Para sustentar o backend do **Portal ITSM Service Desk** com alta manutenibilidade, segurança e alinhamento ao banco de dados PostgreSQL (v9), a estrutura do NestJS deve seguir o padrão **Modular por Domínio (Feature-First)** integrado com conceitos de *Clean Architecture*. Cada módulo funcional (tickets, ativos, SLA, auditoria) encapsula seus próprios Controllers, Services, DTOs e Repositories, garantindo que o isolamento de tenants (**id_cliente**) e as regras de negócio de TI sejam estritamente respeitados.

2. Estrutura Definitiva da Pasta `src/` no NestJS

```
src/
├── config/                # Configurações Globais (TypeORM, JWT, Swagger, Env)
├── common/                # Utilitários, Guards, Interceptors & Decorators Globais
│   ├── decorators/        # @CurrentUser(), @TenantId(), @Roles()
│   ├── filters/           # HttpExceptionHandler, DatabaseExceptionHandler
│   ├── guards/            # JwtAuthGuard, RolesGuard, TenantGuard
│   ├── interceptors/      # LoggingInterceptor, AuditLogInterceptor (JSONB)
│   └── middleware/        # TenantMiddleware (extração e validação do id_cliente)
├── core/                  # Domínio Puro & Regras de Negócio Empresariais
│   ├── domain/           # Entidades Puras (Ticket, Asset, Change, Problem)
│   └── enums/             # TicketStatus, TicketPriority, UserRole, SLAStatus
├── modules/               # Módulos de Negócio Isolados (Feature-First)
│   ├── auth/             # Autenticação JWT, Refresh Token, Passwords
│   ├── tenants/          # Gestão de Clientes Multi-tenant (MSP)
│   ├── users/            # Usuários, Perfis (RBAC), Departamentos
│   ├── tickets/          # Helpdesk & Gestão de Chamados
│   │   ├── dto/          # CreateTicketDto, UpdateTicketDto, QueryTicketDto
│   │   ├── entities/     # Mapeamento ORM (chamados, comentarios, anexos, worklogs)
│   │   ├── tickets.controller.ts # Endpoints REST (/tickets)
│   │   ├── tickets.service.ts  # Regras de negócio de atendimento e transições
│   │   └── tickets.module.ts   # Registro do módulo NestJS
│   ├── sla/              # Motor de SLA (Turnos, Feriados Recorrentes, Pausas)
│   ├── cmdb/             # Gestão de Ativos, Especificações & Dependências
│   ├── itil/             # Problemas (Known Errors) & Mudanças (RFC)
│   ├── approvals/        # Central de Aprovações (Regra XOR Estrita)
│   ├── kb/               # Base de Conhecimento & Avaliação Única por Usuário
│   ├── csat/             # Pesquisas de Satisfação (1 a 5)
│   └── audit/            # Logs de Auditoria Imutáveis (JSONB)
├── database/             # Migrations, Seeds e ORM Providers
└── main.ts               # Bootstrap da Aplicação (ValidationPipe, Swagger, CORS)
```

3. Mapeamento de Camadas & Responsabilidades

Camada / Pasta	Tipo NestJS	Responsabilidade no Sistema ITSM / GLPI
common/guards	Guard	Valida token JWT (JwtAuthGuard) e garante o isolamento multi-tenant (TenantGuard).

Camada / Pasta	Tipo NestJS	Responsabilidade no Sistema ITSM / GLPI
common/interceptors	Interceptor	Captura mutações e grava automaticamente na tabela logs_auditoria (JSONB).
modules/tickets	Module	Gerencia o ciclo de vida dos chamados, anexos (20MB), chat e apontamento de worklogs.
modules/sla	Service	Calcula prazos de SLA descontando turnos, feriados recorrentes e pausas ativas.
modules/cmdb	Controller/Service	Mantém o inventário de hardware/software e mapeia a árvore de dependências.
modules/approvals	Service	Executa a validação de aprovações com a regra de Exclusão Mútua XOR (Chamado/Mudança).
modules/audit	Service/Repository	Interface de leitura para auditoria imutável (bloqueada contra UPDATE/DELETE).

4. Regras Diretivas de Implementação Backend (System Prompt para a IA)

REGRAS INVIOLÁVEIS PARA A INTELIGÊNCIA ARTIFICIAL AO GERAR CÓDIGO NESTJS:

- Filtro de Tenant Obrigatório:** Toda consulta e mutação deve incluir rigorosamente o id_cliente obtido através do @TenantId() proveniente do token JWT validado no TenantGuard.
- Tratamento de Datas (TIMESTAMP TZ):** Utilize objetos Date e bibliotecas de fuso horário (ex: *date-fns-tz* or *dayjs*) convertendo os horários para o fuso_horario cadastrado no expediente.
- Validação DTOs Estrita:** Use *class-validator* e *class-transformer* com `ValidationPipe({ whitelist: true, forbidNonWhitelisted: true })` em todos os controllers.
- Regra XOR nas Aprovações:** Ao criar uma requisição de aprovação, exija exatamente um ID (id_chamado XOR id_mudanca).
- Erros Amigáveis:** Utilize as exceções nativas do NestJS (`NotFoundException`, `ForbiddenException`, `BadRequestException`) traduzidas para o português.